

Metavault

A self-hostable media library for personal records, tracking, search, and recommendations

Nikoloz Kvrivishvili, Guga Akopashvili, Giorgi Ratiani



ILIA STATE UNIVERSITY
Faculty of Business, Technology and Education
School of Technology - Computer Science

SUPERVISOR
Prof. Dr. Erekle Magradze

Metavault is a self-hostable web application for tracking movies, TV shows, anime, games, books, manga, and other media in one place. The core library works without external services: users can create records, search them, organize them, and export their data. Source integrations and AI are opt-in. TMDb, AniList, IGDB, or OpenLibrary can fill metadata, and a local or hosted OpenAI-compatible model can power chat and recommendations.

MOTIVATION AND PRODUCT SCOPE

Purpose of Metavault

People who track several media types usually split their history across separate tools. Ratings, statuses, tags, unfinished items, and backups end up spread across different accounts.

OPT-IN INTEGRATIONS

The library, EZQ, import/export, and account flows do not require metadata sources or an AI provider. Enrichment and assistant features are toggled on by users explicitly.

Data ownership

Library data lives in SQLite. It can be easily exported in bundles in a portable format, so it can be backed up, inspected, or moved.

Shared media model

Movies, shows, anime, games, books, and manga share one record shape for status, ratings, tags, and collections.

Easy enrichment

Configured source APIs can fill titles, covers, dates, public ratings, source IDs, and tags when the user asks for enrichment.

Portable setup

Docker and the open repository make the project easier to run, audit, modify, and reuse.

Features of Metavault

The current app supports the main flows: account creation and management, library management, querying and executing commands with EZQ, entry enrichment, chatting with assistants, getting recommendations, and importing/exporting.

Accounts and access

Register, verify email, sign in, update profile, upload avatar, reset password, and delete the account.

Media library

Create, view, update, delete, rate, tag, upload covers, and track status across supported media types.

Organization

Collections and content nodes for lists, episodes, chapters, volumes, and related links.

Search and EZQ

One input for search, create, update, delete, aliases, enrichment, and catalogue pull commands.

Source enrichment

Use configured TMDb, AniList, IGDB, or OpenLibrary sources to fill missing metadata.

Recommendations

Generate suggestions from catalogue entries, ranked with embeddings, filters, and scoring rules.

Portability

Self-host with Docker, persist data in a SQLite database, and move data through import/export bundles.

AI assistant

Use OpenAI-compatible local or hosted model for chat, discuss about query results, and get recommendations.

SYSTEM DESIGN AND QUERY MODEL

Overview of Architecture

The web client is the primary interface to our server. The server handles all of the functionality and executes instructions generated by ezq. SQLite is used to persist the data.

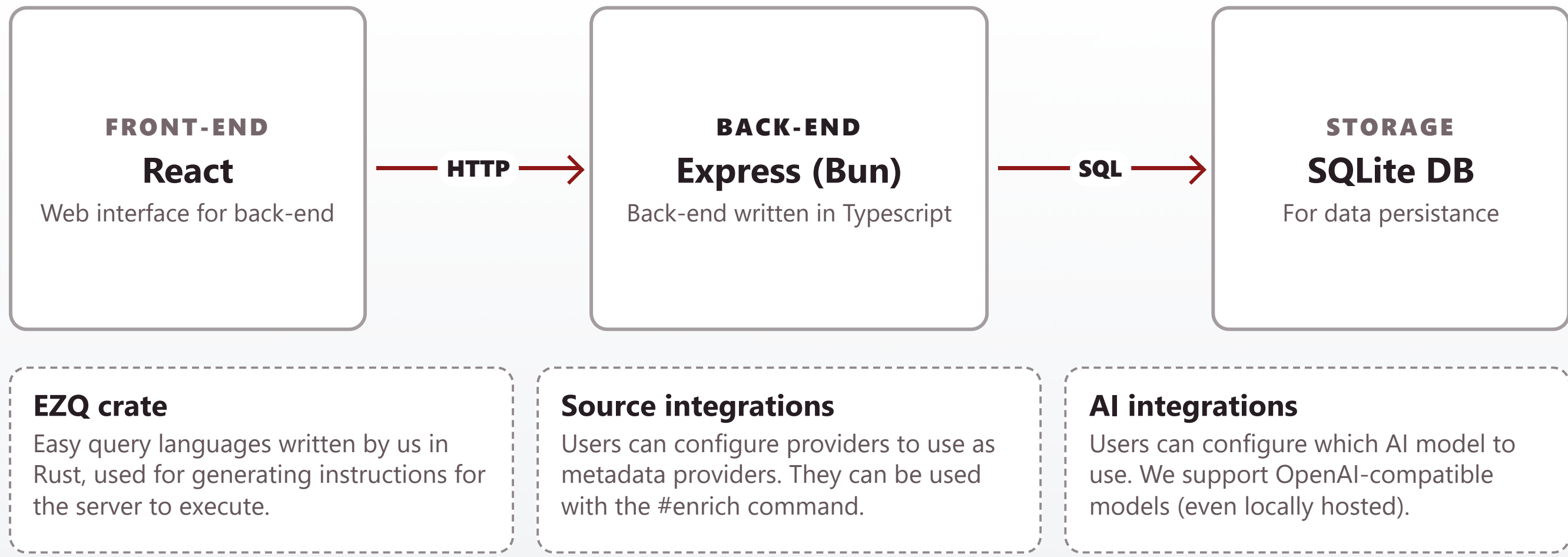


Figure 1. High-level architecture of Metavault.

EZQ (Easy Query)

EZQ is a Rust crate created by us. It gives us the ability to execute most common library management operations with easy syntax.

<code>/s status:in_progress type:movie</code>	Search in-progress movies
<code>/c Dune type:book status:planning</code>	Create an book entry
<code>/u Joker > status:Finished</code>	Update entry progress
<code>/d public_rating:<7</code>	Delete entries

EZQ execution: The web client expands and validates commands for quick feedback using the EZQ crate built for browser environment. The server parses the same query again, builds the list of action, and runs them for the signed-in user using the EZQ crate built for Node.

Figure 2. Example EZQ commands.

INTEGRATIONS, EVIDENCE, AND NEXT STEPS

Enrichment and recommendations

Source integrations can enrich saved entries and build the recommendation catalogue. Recommendation requests embed the prompt, filter catalogue candidates, and rank the matches before the assistant explains them.

Recommendation overview

MetaVault embeds the prompt and compares it with catalogue vectors using cosine similarity. The final score also considers keyword overlap, public rating, and popularity.



The assistant writes the answer after the server returns ranked catalogue records.

Figure 3. Source metadata becomes catalogue candidates before the assistant explains recommendations.

Recommendation study outcomes

Five participants evaluated 25 prompts. Each prompt returned five recommendations, for a total of 125 recorded ratings.

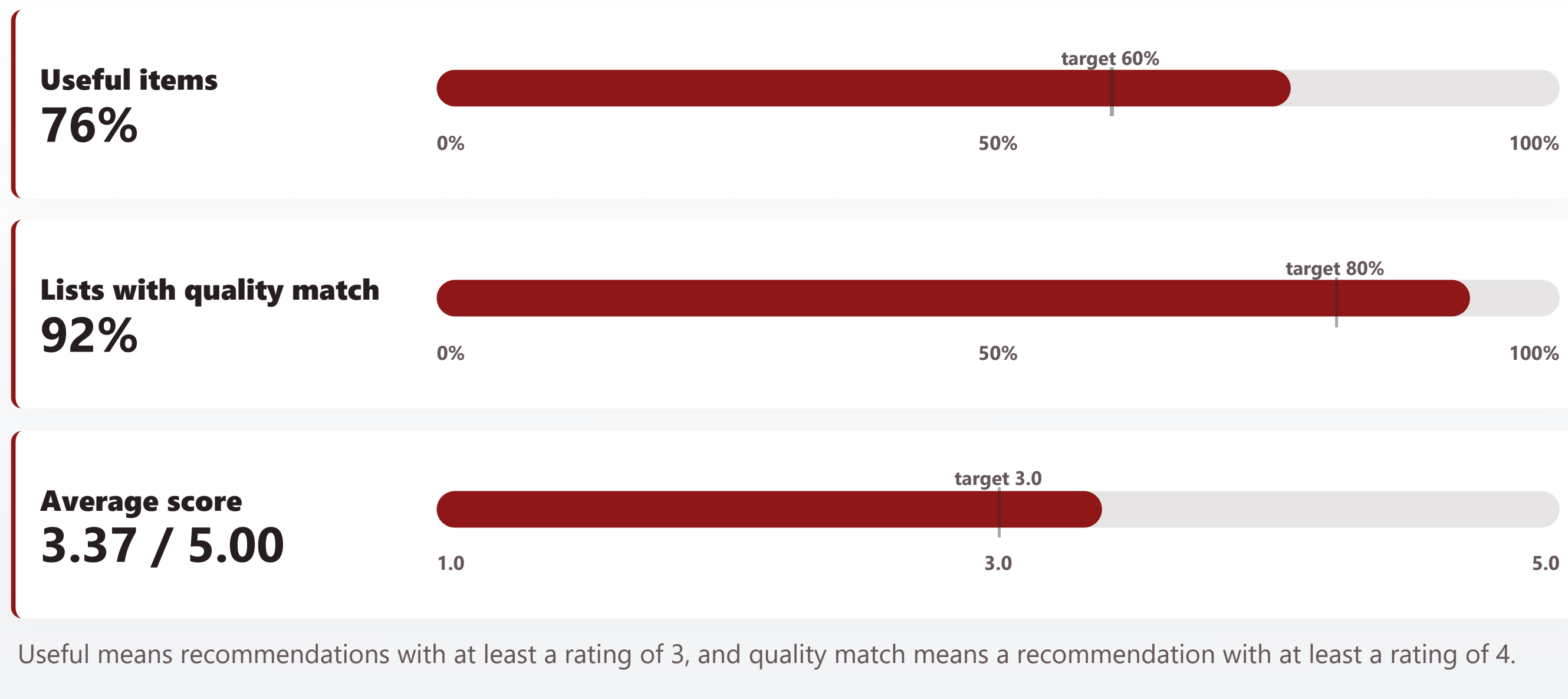


Figure 4. Recommendation study summary.

Future Improvements

Recommendation quality

Tune scoring weights, handle negative constraints better, and utilize a broader catalogue.

Usability improvements

Add release tracking, browser extensions for adding media easier, and a simpler way for less advanced users to query their libraries.

Search vocabulary

Improve EZQ docs and cross-media tag handling for movies, anime, games, books, manga, and shows.

